MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

Department of Information and Communication Technology (ICT)

Attendance Report Analysis

with Dynamic QR and Face Recognition

Submitted by: Md Atikur Rahman

Student ID: IT24026

Department: Information and Communication Technology (ICT)

Institution: Mawlana Bhashani Science and Technology University (MBSTU)

Supervisor: Professor Dr. Ziaur Rahman, Department of ICT, MBSTU

Contents

1 Project Abstract and Summary **3**

2 Problem Statement and Justification **3**

2.1 Problem Definition 3

2.2 Why Existing Solutions Fall Short 4

2.3 Justification 4

3 Project Goals and Objectives **4**

4 Detailed Methodology and System Architecture **5**

4.1 Development Methodology 5

4.2 High-Level Architecture 5

4.3 End-to-End Data Flow 5

4.4 Core Business Rules and Formulas 6

4.5 Database Design (Principal Entities) 7

4.6 Technology Stack and Components 7

4.7 Security Framework 8

4.8 Deployment Architecture (Render) 8

5 Prototype Definition and Deliverable **8**

5.1 Minimum Viable Product (MVP) 8

5.2 Deferred to the Final Build 9

5.3 Final Deliverables 9

6 Risk Assessment and Mitigation Strategies 10

1. Project Abstract and Summary

Attendance Report Analysis is a production-oriented, web-based attendance management and analytics platform designed for university environments. Manual roll calls and spreadsheet-based tracking are slow, error-prone, easy to falsify, and incapable of warning students before they lose exam eligibility. This project replaces them with a unified system built on Python (Flask), SQLAlchemy, and PostgreSQL, deployed on the Render cloud platform.

The system models the real academic structure of a university (department, session, year, semester, batch, section, course offering) so that simultaneously running year levels never mix. Teachers record attendance through one of two independent methods: dynamic, expiring QR codes or face recognition based on stored embeddings. A single centralized attendance engine classifies each student as Present, Late, or Absent (20-minute late threshold), applies a three-consecutive-late penalty, computes effective attendance, and evaluates the 60% exam-eligibility rule. A risk-analysis service classifies students as Safe, Warning, High Risk, or Critical using current, required, and maximum-possible attendance, then triggers idempotent warning emails. Role-based dashboards, CSV/PDF reports, and immutable audit logs complete the platform.

The expected impact is fewer proxy-attendance incidents, earlier student intervention, reduced administrative workload, and transparent, auditable attendance records.

2. Problem Statement and Justification

2.1 Problem Definition

In most Bangladeshi public universities, including many departments within the same institution, attendance is still recorded on paper or in manually maintained spreadsheets. Attendance policy, however, is strict: students below 60% attendance may be barred from final examinations. This creates several specific problems:

1. **Late discovery of risk.** Students typically learn about attendance shortfalls only when eligibility lists are published, when recovery is often mathematically impossible.
2. **Proxy attendance and falsification.** Paper roll calls and static QR codes allow absent students to be marked present by peers.
3. **Inconsistent calculation.** Different teachers apply late-arrival and penalty rules differently, and percentage formulas vary between spreadsheets.
4. **Academic-structure confusion.** Several year levels run the same semester concurrently. Without a structured model, students, teachers, courses, and batches are mixed or double-counted.
5. **No accountability.** Manual corrections leave no trace of who changed what, when, and why.
6. **Administrative overhead.** Teachers spend class time on roll calls, and administrators spend days compiling reports.

2.2 Why Existing Solutions Fall Short

- **Generic LMS attendance plug-ins** record presence but lack university-specific logic such as the three-late penalty, planned versus conducted classes, and recoverability analysis.
- **Static QR attendance apps** are trivially shared through messaging apps, with no expiry, token binding, or audit trail.
- **Commercial biometric systems** are expensive, hardware-dependent, and rarely integrate with an institution's own academic structure.
- **Spreadsheets** provide no access control, no automation, and no reliable audit history.

2.3 Justification

Technical: The project integrates relational modeling, secure token design, machine-learning-based face embeddings, analytics, and cloud deployment, which together demonstrate the full breadth of ICT engineering.

Socio-economic: The platform uses commodity hardware (students' smartphones and a browser), open-source libraries, and low-cost cloud hosting. This makes it affordable for a public university, while giving students a fair, transparent view of their standing and giving administrators trustworthy data.

3. Project Goals and Objectives

Overall goal: To design, implement, test, and deploy a secure and extensible university attendance platform that automates attendance capture, calculation, risk detection, and reporting.

#	Objective	Measurable Success Criterion
1	Implement a normalized academic data model (Course → Course Offering → Teacher Assignment → Student Enrollment) with automated rule enforcement.	Zero cross-batch/semester data mixing in tests; the one-theory-course-per-teacher-per-semester-batch rule is enforced in 100% of test cases.
2	Build two independent attendance methods (dynamic QR and face recognition) behind pluggable interfaces.	QR tokens expire correctly and reject duplicates; face verification achieves at least 90% true-accept rate with a false-accept rate below 2% on a pilot test set of at least 30 volunteers.
3	Develop centralized engines for Present/Late/Absent classification, the three-late penalty, attendance percentage, 60% eligibility, and risk analysis.	All engine functions pass at least 40 automated unit tests, including boundary cases (exactly 60%, exactly 20 minutes late, planned 35 / conducted 24 → max possible 68.57%).

#	Objective	Measurable Success Criterion
4	Deliver role-based dashboards, CSV/PDF reports, automated emails, audit logging, and a Render deployment.	Public HTTPS deployment with a working health check; idempotent cron jobs that produce no duplicate emails across repeated runs; every manual correction recorded in the audit log.

4. Detailed Methodology and System Architecture

4.1 Development Methodology

The project follows a **design-first, iterative** approach: (1) complete requirements, architecture, and database design; (2) implement in vertical modules, with each module's purpose, dependencies, and future impact documented before coding; (3) test-driven validation of every business rule; (4) continuous integration through Git; (5) deployment to Render and pilot evaluation. Business rules are never duplicated: each lives in exactly one service.

4.2 High-Level Architecture

The system is a layered, modular Flask application:

1. **Presentation layer:** Jinja2 templates with HTML, CSS, and JavaScript. Separate Admin, Teacher, Student, and Auth views with a sidebar layout, searchable and paginated tables, charts, and a mobile-friendly attendance flow.
2. **Route layer:** Thin Flask blueprints that handle HTTP, validate input, check authorization, and delegate to services. No business logic lives here.
3. **Service layer:** All business rules in dedicated services: attendance engine, late/penalty service, eligibility service, risk service, enrollment and assignment services, routine service, audit service.
4. **Integration modules:** `qr/` (token generation and validation), `face/` (replaceable embedding interface), `email/` and `notifications/` (SMTP with EmailLog), `reports/` (CSV/PDF), `analytics/` (Pandas aggregation).
5. **Data layer:** SQLAlchemy ORM with Flask-Migrate (Alembic). SQLite for development; Render PostgreSQL for production, selected through the `DATABASE_URL` environment variable.
6. **Infrastructure layer:** Gunicorn web service, Render Cron Jobs for background work, WhiteNoise for static files, and stdout logging.

4.3 End-to-End Data Flow

Setup. Admin creates departments, sessions, years, semesters, batches, courses, and course offerings. Teachers are marked eligible for courses, then assigned to offerings. Students are

enrolled in offerings for their current semester. Admin builds routines linking offering, teacher, day, time, room, and batch.

Reminder. A cron job scans the routine and emails the teacher at the configured lead time, using EmailLog to prevent duplicates.

Session. The teacher starts an attendance session (Scheduled → Active) and selects QR or Face as the method.

Capture (QR). The system issues a short-lived, signed token rendered as a QR code that refreshes periodically. The student scans it while logged in. The server validates the session, token, expiry, enrollment, and duplicates.

Capture (Face). The student's browser captures a frame over HTTPS. The face service generates an embedding and compares it with the stored profile. A match records attendance; a failure is logged but does **not** mark the student absent.

Classification. The engine compares arrival time with class start: before the 20-minute threshold = Present; at or after = Late.

Finalization. When the session closes, an idempotent job marks students without valid records as Absent.

Computation. The attendance engine applies the three-consecutive-late penalty and calculates effective attendance against conducted classes. The eligibility and risk services then classify each student.

Notification. Students entering a risk state receive one warning email per risk event, and results appear on dashboards and reports.

Audit. Every sensitive action (correction, assignment change, session start/close, email) is written to the audit log.

4.4 Core Business Rules and Formulas

- **Attendance %** $= EffectivePresentClassesTotalConductedClasses \times 100$
- **Late rule:** Arrival before start + threshold = Present; otherwise Late. The threshold is a configurable system or course setting.
- **Penalty rule:** Three consecutive Late records in one course produce one penalty. Original records are never altered; actual status, late count, consecutive-late count, penalty count, and effective attendance are tracked separately.
- **Eligibility rule:** Attendance $\geq 60\%$ means Exam Eligible, and exactly 60% is eligible.
- **Maximum possible attendance:** $Effectivepresent + RemainingclassesPlannedclasses \times 100$. Example: planned 35, conducted 24, effective present 13, remaining 11 → $(13 + 11) \div 35 = 68.57\%$.
- **Recoverability:** If perfect attendance in all remaining classes cannot reach 60%, the student is flagged CRITICAL / NOT RECOVERABLE.
- **Risk levels:** SAFE, WARNING, HIGH_RISK, and CRITICAL, with configurable and documented thresholds.

4.5 Database Design (Principal Entities)

Users, Roles, Departments, AcademicSessions, Years, Semesters, Batches, Students, Teachers, Courses, CourseOfferings, TeacherCourseEligibility, TeacherAssignments, StudentEnrollments, Routines, Classes, AttendanceSessions, Attendance, LateRecords, AttendancePenalties, FaceProfiles, FaceVerificationLogs, QRSessionTokens, RiskAlerts, EmailLogs, Notifications, AuditLogs, and SystemSettings.

Each table has a primary key, foreign keys, constraints, indexes, and created/updated timestamps. Students, teachers, and courses are never linked directly; all relationships pass through enrollment, assignment, and course-offering tables. Unique constraints (for example, one attendance record per student per session) provide database-level idempotency.

4.6 Technology Stack and Components

Component	Choice	Purpose
Backend	Python 3, Flask, Blueprints	Web application and APIs
ORM / Migrations	SQLAlchemy, Flask-Migrate	Database abstraction and schema versioning
Database	SQLite (dev), Render PostgreSQL (prod)	Persistent relational storage
Authentication	Flask-Login, Werkzeug password hashing, Flask-WTF (CSRF), Flask-Limiter	Sessions, RBAC, brute-force protection
QR	`qrcode` library plus signed tokens (`itsdangerous`)	Dynamic, expiring QR generation and validation
Face	Lightweight ONNX-based embedding model (e.g., InsightFace or an equivalent CPU model) behind a `FaceRecognizer` interface	Face embeddings without heavy compilation such as dlib; final choice validated against Render build and memory limits
Analytics	Pandas; Chart.js	Aggregation and interactive charts
Reports	CSV module; ReportLab or WeasyPrint	CSV and PDF export
Email	SMTP via environment variables	Reminders and warnings, logged in EmailLog
Server	Gunicorn, WhiteNoise	Production serving on Render
Deployment	`render.yaml` Blueprint, `build.sh`, Render Cron Jobs	Reproducible cloud deployment
Testing	Pytest, pytest-cov	Automated verification

4.7 Security Framework

Password hashing, CSRF protection, ORM-parameterized queries, server-side validation and authorization, secure and expiring session cookies, rate limiting, signed expiring QR tokens, duplicate-attendance prevention, immutable audit logs, and HTTPS. All secrets (secret key, SMTP credentials, QR signing secret, face-data encryption key) live in Render environment variables and never in source code. Face embeddings are stored encrypted in PostgreSQL (the Render filesystem is ephemeral), accessible only to Admin and system processes. The QR security roadmap (V1 dynamic QR through V8 QR + Face + Liveness) is supported by the token and verification interfaces, but the system does **not** claim QR attendance is fully cheat-proof.

4.8 Deployment Architecture (Render)

- **Web Service:** Gunicorn bound to `$PORT`, built by `build.sh`, with `flask db upgrade` as a pre-deploy step.
- **PostgreSQL:** Managed database, connection string injected as `DATABASE_URL`, with backups enabled.
- **Cron Jobs:** (a) class reminder emails, (b) session auto-finalization and absence marking, (c) risk recalculation and warning emails. All are idempotent and independent of the web process.
- **Operations:** `/health` endpoint for health checks, stdout logging, optional Sentry, UTC storage with configurable local-timezone display, and free TLS (required for camera access).

5. Prototype Definition and Deliverable

5.1 Minimum Viable Product (MVP)

The MVP is a working, deployed web application that demonstrates the full attendance lifecycle for one department, one academic session, and at least two concurrent year-semester groups, using seeded realistic data.

Features included in the MVP:

- Authentication with three roles (Admin, Teacher, Student), CSRF protection, and role-based access.
- Admin management of departments, sessions, batches, students, teachers, courses, and course offerings.
- Teacher eligibility, assignment with enforcement of the one-theory-course rule, and student enrollment with duplicate and out-of-semester blocking.
- Routine creation linked to course offerings and teachers.
- Attendance sessions (Scheduled, Active, Closed) with **dynamic QR attendance** using short-lived signed tokens, expiry checks, and duplicate prevention.

- **Face recognition attendance** with admin-managed registration, embedding storage in the database, verification logging, and a failed-match-is-not-absent rule.
- Centralized attendance engine: Present/Late/Absent, three-late penalty, effective attendance, percentage, 60% eligibility, maximum possible attendance, required future attendance, and risk levels.
- Admin correction workflow with a mandatory reason and audit log entry.
- Automatic class-reminder and risk-warning emails with duplicate prevention through EmailLog.
- Basic dashboards for all three roles with charts, plus CSV export for core reports.
- Automated test suite covering the rules listed in the specification.
- Render Blueprint deployment with PostgreSQL, cron jobs, and a health check.

5.2 Deferred to the Final Build

Deferred Feature	Rationale
Location and device validation	Requires additional privacy design and mobile testing
Combined QR + Face and QR + Face + Liveness	Architecture will permit it; implementation follows face-accuracy evaluation
Full PDF report suite for every report type	CSV is prioritized first
Advanced analytics (teacher workload, cross-batch comparison, long-term trends)	Depends on accumulated real data
Bulk CSV import of students and courses	Convenience feature after core workflows stabilize
Password reset by email (full workflow)	Architecture prepared; UI and token flow completed later
Sentry integration and performance tuning	Applied after pilot feedback
Native mobile app	Out of scope; the responsive web app serves mobile browsers

5.3 Final Deliverables

1. Source code repository with documentation and `.env.example`
2. Deployed, HTTPS-accessible application on Render
3. Architecture and database design document (ERD, data-flow diagrams)
4. Automated test suite with coverage report

5. Deployment and security-checklist guide
6. Pilot evaluation report and final presentation

6. Risk Assessment and Mitigation Strategies

#	Risk	Likelihood / Impact	Mitigation and Backup Plan
1	**Face-recognition accuracy and infrastructure limits.** Lighting, camera quality, and similar faces may reduce accuracy, and heavy ML libraries may exceed Render's build or memory limits.	Medium / High	Use a lightweight ONNX/CPU embedding model behind a replaceable `FaceRecognizer` interface, and benchmark candidate models in Month 3 before committing. Tune the similarity threshold on pilot data. **Backup:** If the model cannot run within Render's limits, fall back to a smaller model, or run face attendance as an assisted flow in which failed matches are routed to teacher/admin review. QR attendance remains fully independent and operational.
2	**Free-tier and cloud-service limitations.** Render's free web service spins down when idle, free database instances can expire, and delayed cron or SMTP delivery can affect time-sensitive reminders and QR sessions.	High / Medium	Design cron jobs to be idempotent and tolerant of late execution. Use a keep-alive health check during demonstrations, schedule regular database backups, and keep reminder lead times configurable. **Backup:** Upgrade to a paid Render tier for the evaluation period, or deploy the identical Docker/Gunicorn configuration on a university server. Use a transactional email provider if SMTP limits are hit.

#	Risk	Likelihood / Impact	Mitigation and Backup Plan
3	**Scope and schedule overrun.** The specification is broad (academic structure, two attendance methods, analytics, reports, deployment), and integration defects could delay delivery.	Medium / High	Follow the design-first plan, build in vertical slices, and freeze the MVP scope at the end of Month 2. Use automated tests to catch regressions early and hold weekly supervisor reviews. **Backup:** Move deferred items (PDF reports, advanced analytics) to future work and prioritize the attendance engine, QR, and risk analysis, as these are the core contribution.

Additional operational risks, including privacy of face data and the unavailability of real student volunteers, will be handled through encrypted storage, informed-consent forms, anonymized evaluation data, and seeded synthetic data as a fallback.